

DECISION TREES

A Decision Tree is a supervised machine learning algorithm that is used for both classification and regression tasks. It models decisions by splitting the data into branches based on feature values, forming a tree-like structure where each internal node represents a decision on a feature, each branch represents an outcome of the decision, and each leaf node represents a final prediction or outcome.

Important features of decision trees are –

1. Non-Parametric model: A decision tree is considered a non-parametric model because it doesn't assume a specific mathematical form for the relationship between inputs and outputs.
2. White Box model: A decision tree is often called a "white box" model because its decision-making process is easy to understand and interpret.
3. It is a giant if-else based model.

Why Do We Need Entropy, Information Gain & Gini Index?

A decision tree works by splitting data into smaller and purer subsets.

At every node, the tree must answer:

"Which feature and which split produces the purest child nodes?"

To answer this we use -

- Entropy → measures randomness
- Gini Index → measures impurity
- Information Gain → measures impurity reduction after a split

Symbol Meaning

- S - Original dataset
- N - Number of classes
- P_i - Probability of class i in dataset S
- A – Attribute used for splitting

Entropy vs. Information Gain vs. Gini Index in Decision Trees

Which Split Creates the Purest Child Nodes?

ENTROPY

Measures Disorder

$$\text{Entropy}(S) = -\sum_{i=1}^c p_i \log_2(p_i)$$

- ✓ Entropy = 0 → Pure
- ✓ High Entropy → Impure
- ✓ ID3 Algorithm

INFORMATION GAIN

Reduction in Entropy

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{i=1}^c \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

- ✓ High IG → Better Split
- ✓ IG = 0 → No Gain
- ✓ ID3, C4.5

GINI INDEX

Measures Impurity

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

- ✓ Gini = 0 → Pure
- ✓ High Gini → Impure
- ✓ CART Algorithm

SPLIT SELECTION RULES

Maximize INFORMATION GAIN:

$$\text{Best Split} = \text{Max IG}(S, A)$$



Minimize GINI AFTER SPLIT:

$$\text{Gini}_{\text{split}}(A) = -\sum_{i=1}^c \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

$$\text{Best Split} = \text{Min Gini}_{\text{split}}$$

QUICK COMPARISON

	ENTROPY	INFO GAIN	GINI INDEX
MEASURES	Disorder	Entropy Reduction	Impurity
FORMULA	$-\sum_{i=1}^c p_i \log_2(p_i)$	$\text{Entropy}(S) - \sum_{i=1}^c \frac{ S_v }{ S } \text{Entropy}(S_v)$	$1 - \sum_{i=1}^c p_i^2$
RANGE	0 to $\log_2(c)$	≥ 0	0 to ~0.5
USED IN	ID3	ID3, C4.5	CART

Goal: Find the Split that Maximizes Purity of Child Nodes.

Gini Index After a Split

Suppose a split divides node t into:

- Left child t_L
- Right child t_R

Then the Gini after split is:

$$\text{Gini}_{\text{split}} = \frac{N_L}{N} \text{Gini}(t_L) + \frac{N_R}{N} \text{Gini}(t_R)$$

where:

- N = total samples in parent node
- N_L = samples in left child
- N_R = samples in right child

Various algorithms to create Decision Trees are –

Top Decision Tree Algorithms

Entropy, Information Gain & Gini Index Explained

ID3 (Iterative Dichotomiser 3)

- **Task Type:** Classification only
- **Split Type:** Categorical
- **Split Criterion:** Entropy, Information Gain

Handles Continuous Features: ✗ ✗

Handles Missing Values: ✗ ✗

Pruning Overfitting Control: ✗ ✗

• Key Strengths:

- ✓ Simple and intuitive
- ✓ Good for small datasets

⚠ Key Limitations: No pruning – overfitting risk

- Cannot handle continuous features or missing values
- Highly biased towards features with many variants

C4.5

- **Task Type:** Classification only
- **Split Type:** Categorical & Continuous
- **Split Criterion:** Gain Ratio

Handles Continuous Features: ✓ ✓

Handles Missing Values: ✓ ✓

Pruning Overfitting Control: ✓ ✓

• Key Strengths:

- ✓ Handles both categorical and continuous features
- ✓ Supports missing data
- ✓ Gain ratio adjusts for feature bias

⚠ Key Limitations: Pruning method can remove useful subtrees

- ✓ Computationally more intensive than ID3

CART (Classification & Regression Trees)

- **Task Type:** Classification & Regression
- **Split Type:** Categorical & Continuous
- **Split Criterion:** Gini Index (classif.)

Handles Continuous Features: ✓ ✓ Yes

Handles Missing Values: ✓ ⚠ Yes

Pruning Overfitting Control: ✓ ✓ Yes

• Key Strengths:

- ✓ Handles both classification and regression tasks
- ✓ Computationally efficient (*Gini faster than Entropy*)
- ✓ Uses cost complexity pruning

⚠ Key Limitations:

- ✓ Pruning method can remove useful subtrees
- ✓ Computationally more intensive than ID3

CHAID (Chi-squared Automatic Interaction Detection)

- **Task Type:** Classification & Regression
- **Split Type:** Categorical
- **Split Criterion:** Chi-square Test

Handles Continuous Features: ✗ ✓ Yes

Handles Missing Values: ✓ ✓ Yes

Pruning Overfitting Control: ✓ ✓ Yes

• Key Strengths:

- ✓ Handles categorical features well
- ✓ Handles missing data via merging categories
- ✓ Finds multi-way splits for interpretability

⚠ Key Limitations:

- ✓ Assumes normally distributed data
- ✓ Not ideal for continuous features
- ✓ Overly sensitive to outliers

CHAID (Chi-squared Automatic Interaction Detection)

- **Task Type:** Classification & Regression

HQUEST (Histogram-Quest)

- **Task Type:** Classification & Regression

ID3 ALGORITHM

ID3 (Iterative Dichotomiser 3) is a greedy, top-down decision tree learning algorithm used for classification, which constructs a tree by recursively selecting the attribute that maximizes Information Gain (based on entropy) at each node.

Steps of the ID3 Algorithm (Working) are -

1. Start with the full training dataset
2. Calculate entropy of the target class
3. For each attribute:
 - Partition the dataset based on attribute values
 - Compute entropy for each partition
 - Compute Information Gain
4. Select attribute with highest Information Gain
5. Create a decision node for that attribute
6. Repeat recursively for each branch
7. Stop when:
 - All samples belong to one class
 - No attributes remain
 - Dataset is empty (use majority class)

Problem Statement

Predict whether a person will Play Tennis based on weather conditions.

Dataset –

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Rain	Mild	High	Strong	No

Step 1 Calculate Entropy of Target Variable

- Yes = 9

- No = 5

$$\begin{aligned}\text{Entropy}(S) &= -\left(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14}\right) \\ &= -(0.643 \times -0.64 + 0.357 \times -1.485) \\ &= 0.94\end{aligned}$$

Step 2 Compute Information Gain for Each Attribute

◆ Attribute: Outlook

Value	Yes	No	Entropy
Sunny	2	3	0.971
Overcast	4	0	0
Rain	3	2	0.971

$$\begin{aligned}\text{Entropy after split} &= \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.693 \\ \text{IG}(\text{Outlook}) &= 0.94 - 0.693 = 0.247\end{aligned}$$

Attribute: Temperature

$$\text{IG}(\text{Temperature}) = 0.029$$

◆ Attribute: Humidity

$$\text{IG}(\text{Humidity}) = 0.151$$

◆ Attribute: Wind

$$\text{IG}(\text{Wind}) = 0.048$$

Step 3 Select Best Attribute

Attribute	Information Gain
Outlook	0.247 (Highest)
Humidity	0.151
Wind	0.048
Temperature	0.029

☑ Root Node = Outlook

Step 4 ☐ Recursive Splitting

🌿 Outlook = Overcast

- All examples = Yes
 - Leaf node = Yes
-

🌿 Outlook = Sunny

Subset:

Humidity	PlayTennis
----------	------------

High	No
------	----

High	No
------	----

High	No
------	----

Normal	Yes
--------	-----

Normal	Yes
--------	-----

Split on Humidity → pure nodes

🌿 Outlook = Rain

Subset:

Wind	PlayTennis
------	------------

Weak	Yes
------	-----

Weak	Yes
------	-----

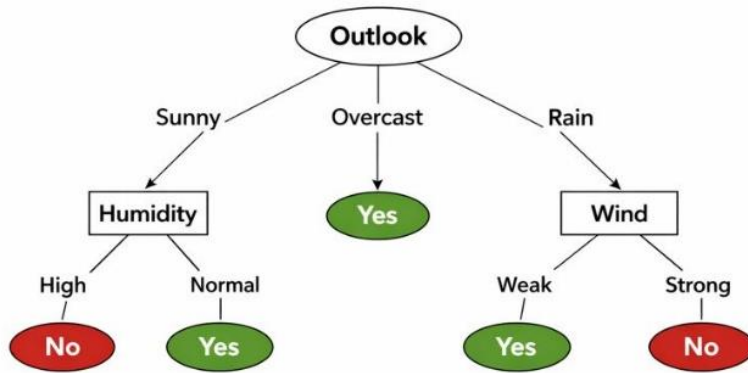
Weak	Yes
------	-----

Strong	No
--------	----

Strong	No
--------	----

Split on Wind → pure nodes

Final Decision Tree -



CART ALGORITHM

CART (Classification and Regression Trees) is a binary decision tree learning algorithm that builds trees by recursively selecting the feature and threshold that minimize node impurity, using Gini Index for classification and Mean Squared Error (MSE) for regression.

4.1 CART Algorithm — Step-by-Step Working

General Procedure

1. Start with the full dataset at the root
2. For each feature:
 - Try all possible split thresholds
 - Compute impurity of resulting binary splits
3. Select the split with lowest impurity
4. Create left and right child nodes
5. Recursively repeat for each child
6. Stop when:
 - Node becomes pure
 - Minimum samples reached
 - Maximum depth reached
7. Apply cost-complexity pruning to remove weak branches

Problem Statement

Predict whether a person will Buy a Computer.

Dataset

Age	Income	Student	Credit	Buy
Youth	High	No	Fair	No
Youth	High	No	Excellent	No
Middle	High	No	Fair	Yes

Age	Income	Student	Credit	Buy
Senior	Medium	No	Fair	Yes
Senior	Low	Yes	Fair	Yes
Senior	Low	Yes	Excellent	No
Middle	Low	Yes	Excellent	Yes
Youth	Medium	No	Fair	No
Youth	Low	Yes	Fair	Yes
Senior	Medium	Yes	Fair	Yes
Youth	Medium	Yes	Excellent	Yes
Middle	Medium	No	Excellent	Yes
Middle	High	Yes	Fair	Yes
Senior	Medium	No	Excellent	No

Step 1 □ Gini Impurity of Root Node

- Yes = 9
- No = 5

$$\begin{aligned} \text{Gini}(S) &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\ &= 1 - (0.413 + 0.127) = 0.46 \end{aligned}$$

Step 2 □ Evaluate Possible Splits

◆ Split on Age

CART forces binary splits, so multi-category features are split as subsets.

Example split:

- Left: {Youth}
- Right: {Middle, Senior}

Left (Youth)

- Yes = 2, No = 3

$$\text{Gini}_L = 1 - (0.4^2 + 0.6^2) = 0.48$$

Right (Middle + Senior)

- Yes = 7, No = 2

$$\text{Gini}_R = 1 - (0.78^2 + 0.22^2) = 0.35$$

Weighted Gini

$$\text{Gini}_{\text{split}} = \frac{5}{14}(0.48) + \frac{9}{14}(0.35) = 0.40$$

◆ Split on Student (Yes / No)

Student = Yes

- Yes = 6, No = 1

$$\text{Gini}_L = 1 - (0.86^2 + 0.14^2) = 0.24$$

Student = No

- Yes = 3, No = 4

$$\text{Gini}_R = 1 - (0.43^2 + 0.57^2) = 0.49$$

Weighted Gini

$$\text{Gini}_{\text{split}} = \frac{7}{14}(0.24) + \frac{7}{14}(0.49) = 0.36$$

◆ Split on Credit Rating

Weighted Gini ≈ 0.43

Step 3 Choose Best Split

Attribute Weighted Gini

Age 0.40

Student 0.36 (Lowest)

Credit 0.43

☑ Root Node = Student

Step 4 Recursive Splitting

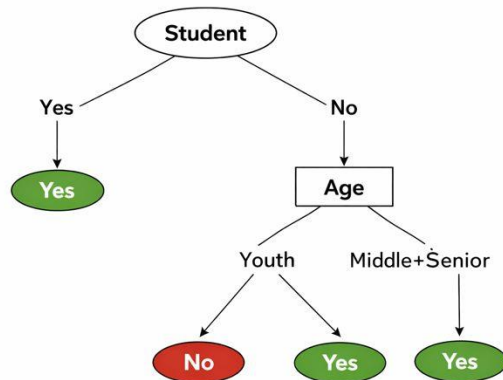
✂ Student = Yes

Mostly Yes, stop or split further if needed.

✂ Student = No

Split further on Age or Credit using same procedure.

Final CART Tree



Pruning in CART (Cost-Complexity)

CART uses:

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

Where:

- $R(T)$ = misclassification error
- $|T|$ = number of leaf nodes
- α = complexity parameter

Prunes subtrees that do not sufficiently reduce error.

Difference between gini impurity and entropy

Pre – Pruning and Post – Pruning

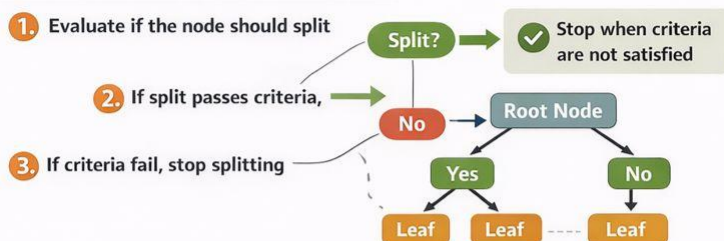
Pruning is the process of removing unnecessary branches to reduce overfitting, improve generalization, and simplify the tree.

Reasons to pruning –

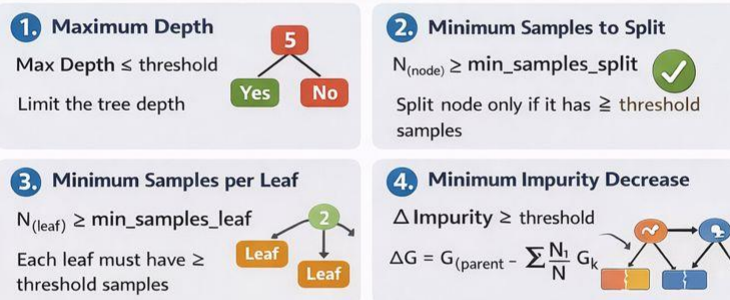
- ❑ Prevents overfitting – Pruning removes overly specific branches so the tree generalizes better to unseen data.
- ❑ Improves interpretability – A pruned tree is smaller and simpler, making decision rules easier to understand and explain.
- ❑ Reduces variance and noise learning – By eliminating splits based on small or noisy samples, pruning stabilizes predictions.

Pre-Pruning in Decision Trees

How Pre-Pruning Works



Common Pre-Pruning Techniques



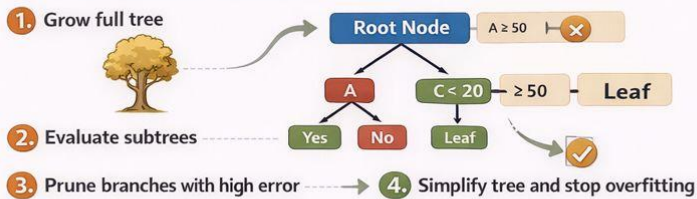
When to Use Pre-Pruning



💡 **TIP:** Pre-Pruning places constraints during tree building to stop growth early and prevent overfitting.

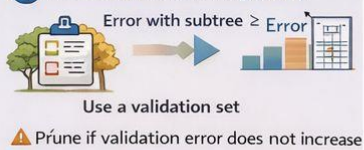
Post-Pruning in Decision Trees

How Post-Pruning Works

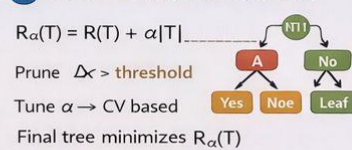


Common Post-Pruning Techniques

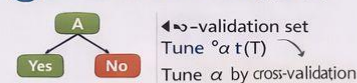
1. Reduced Error Pruning (REP)



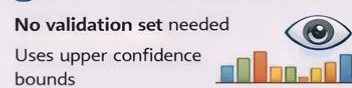
3. Cost-Complexity Pruning (CART)



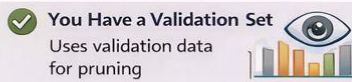
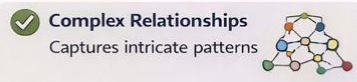
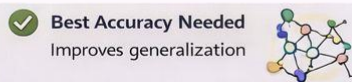
2. Pessimistic Error Pruning (C4.5)



4. Pessimistic Error Pruning (C4.5)



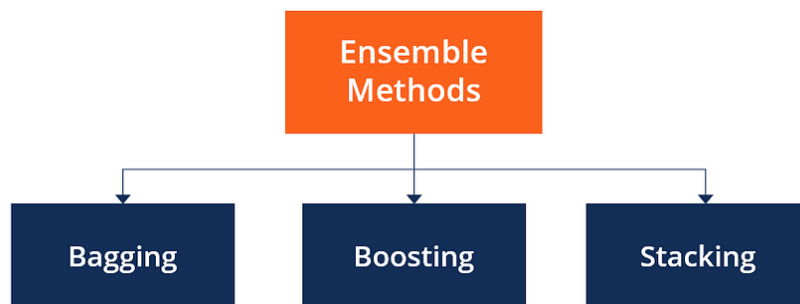
When to Use Post-Pruning



💡 **TIP:** Post-Pruning first grows a full tree, then removes unnecessary branches to reduce overfitting. It's slower but usually more effective.

Ensemble Learning

- Ensemble learning is a machine learning technique that combines multiple individual models (called base models or weak learners) to create a stronger, more accurate model.
- It is based on the idea that a group of weak learners can work together to make better predictions than a single strong learner.
- The individual models in an ensemble can be trained using different algorithms or with the same algorithm but on different subsets of the training data.
- Types of Ensemble Methods are -

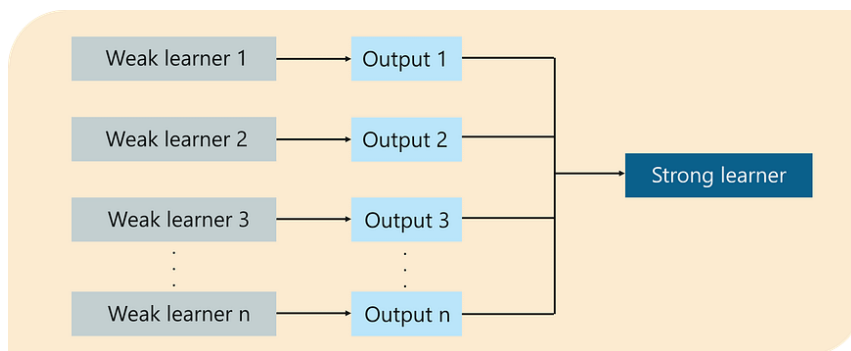


Bagging:

1. Bagging, which stands for Bootstrap Aggregating and the key idea behind bagging is that by training multiple models on different bootstrap samples, the ensemble can effectively reduce the variance of the predictions. The individual models may have high variance or overfit the training data, but when combined, they tend to produce more stable and accurate predictions. Additionally, bagging can help in reducing the impact of outliers or noisy data points.
2. It aims to improve the accuracy and stability of supervised learning models by combining the predictions of multiple base models trained on different subsets of the training data.
3. Popular Bagging algorithm is Random Forest algorithm.

Boosting:

1. Boosting is a machine learning ensemble technique that combines multiple weak or base models typically referred to as “weak learners” or “base learners” to create a strong predictive model.
2. Boosting works by training a sequence of weak models, where each new model focuses on different aspects of the data and also try to correct the errors of the previous ones. By combining these complementary learners, boosting achieves higher overall performance, as seen in algorithms like AdaBoost and Gradient Boosting used across classification, regression, and ranking tasks.



RANDOM FOREST

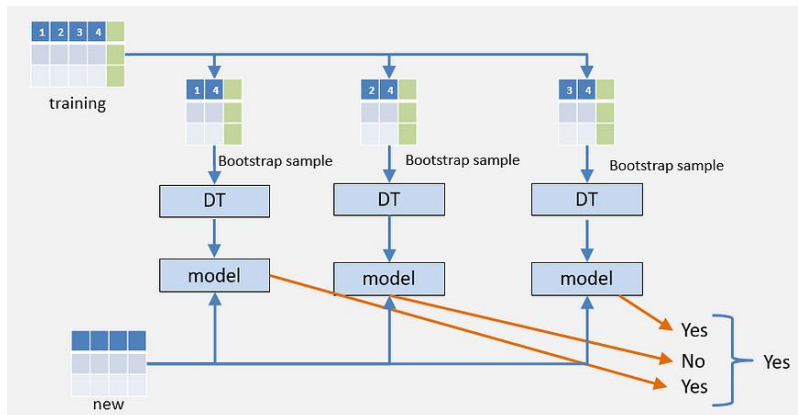
Random forest is built on two basic concepts: bagging and random subspace:

- Bagging (Bootstrap Aggregating): It is the process of creating multiple subsets of the training data through random sampling with replacement and using these subsets to train a collection of decision trees. The combination of these trees' predictions results in a more accurate and less overfitting-prone model.
- Random Subspace: It refers to the technique of randomly selecting a subset of features (variables or attributes) from the original feature set for each individual decision tree within the ensemble. By allowing each tree to focus on a different subset of features, the Random Forest can capture different patterns and relationships within the data, leading to more accurate and stable predictions, especially when dealing with high-dimensional datasets or datasets with many irrelevant features.

What is a Random Forest?

A random forest is essentially a collection (or “forest”) of decision trees trained on different subsets of the data. Each tree in the forest is trained independently, and the final prediction is made by aggregating the predictions of all trees, either by taking the majority vote in classification or averaging the predictions in regression.

How Random Forests Work ?



The random forest algorithm operates in the following steps:

1. Bootstrap Sampling (Bagging):

- From the original training dataset, multiple bootstrap samples are generated. A bootstrap sample is a random sample drawn with replacement, meaning that the same data point may appear more than once in a sample, while others may not appear at all.
- Bootstrap sampling simply means sampling rows at random from the training dataset with replacement.

2. Random Feature Selection:

- Instead of considering all features (as in standard decision trees), the algorithm randomly selects a subset of features. This adds additional randomness because correlated predictors cannot help in improving the accuracy of prediction.
- This random feature selection is a key difference between random forests and bagging with decision trees.
- We do so to avoid the correlation between the trees. Suppose that we have a very strong predictor in the data set along with a number of other moderately strong predictors, then in the collection of bagged trees, most or all of our decision trees will use the very strong predictor for the first split! All bagged trees will look similar. Hence all the predictions from the bagged trees will be highly correlated.

3. Grow Decision Trees:

- Each tree in the forest is grown to the maximum extent without pruning. The goal is to let each tree overfit its particular bootstrap sample.
- Each decision tree in the random forest is trained on a different bootstrap sample.

4. Aggregation:

- For classification, the predictions of all trees are combined using majority voting. The class that gets the most votes becomes the final prediction.
- For regression, the predictions of all trees are averaged to produce the final output.

Why use Random Forest instead of a Decision Tree ?

1. Improved accuracy

Random Forest usually achieves higher accuracy than a single Decision Tree. By averaging predictions from many trees, it reduces both bias and variance.

2. Reduced overfitting

Decision Trees often overfit by learning noise and outliers.

Random Forest combines multiple trees, producing a more stable and generalized model.

3. Handling high-dimensional data

Random Forest works well with many input features without extra feature selection. Decision Trees can become overly complex and overfit in high-dimensional spaces.

4. Feature importance

Random Forest provides feature importance scores based on impurity reduction. This helps identify which variables contribute most to model performance.

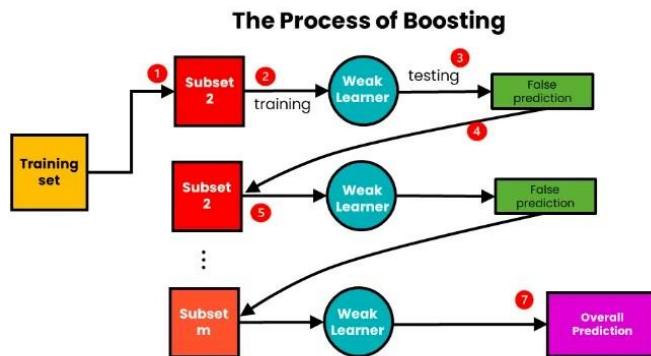
5. Resistance to noise and outliers

Individual Decision Trees are sensitive to noisy data and outliers.

Random Forest minimizes their impact through ensemble averaging.

BOOSTING ALGORITHMS

Boosting combines high-bias weak learners through sequential training to reduce both bias and variance by focusing more on previously misclassified data points, unlike bagging which trains models in parallel.



Ada BOOST ALGORITHM

AdaBoost (Adaptive Boosting) is an ensemble learning algorithm that builds a strong classifier by sequentially combining multiple weak learners, typically decision stumps, where each learner focuses more on the samples misclassified by previous ones through adaptive reweighting, and final predictions are made using weighted majority voting.

Working steps are –

Step 1: Initialize sample weights

At the beginning, all training samples are assigned equal weights:

$$w_i = \frac{1}{N}$$

where N is the total number of samples.

Step 2: Train a weak learner (Decision Stump)

Train a weak classifier $h_t(x)$ (usually a decision stump) on the entire dataset using the current sample weights.

The stump is chosen such that it minimizes the weighted classification error.

Step 3: Compute weighted error

Calculate the total weighted error of the weak learner:

$$\epsilon_t = \sum_{i=1}^N w_i \cdot \mathbb{1}(y_i \neq h_t(x_i))$$

This represents the sum of weights of the misclassified samples.

Step 4: Compute learner weight (Amount of Say)

Calculate the contribution (performance) of the weak learner:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- Lower error $\rightarrow$ higher α_t
- If $\epsilon_t = 0.5$, the learner has no contribution

Step 5: Update sample weights

Update the sample weights to emphasize incorrectly classified samples:

$$w_i \leftarrow w_i \cdot e^{-\alpha_t y_i h_t(x_i)}$$

Which results in:

- Incorrectly classified samples:

$$w_i \leftarrow w_i \cdot e^{+\alpha_t}$$

- Correctly classified samples:

$$w_i \leftarrow w_i \cdot e^{-\alpha_t}$$

Step 6: Normalize the weights

Normalize the weights so that they sum to 1:

$$w_i \leftarrow \frac{w_i}{\sum_{i=1}^N w_i}$$

Step 7: Repeat for multiple rounds

Repeat Steps 2 to 6 for $t = 1$ to T to build multiple weak learners, where each new learner focuses more on previously misclassified samples.

Step 8: Final strong classifier

Combine all weak learners using a weighted majority vote:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

If problem in understanding above steps read this

(<https://medium.com/@pingsubhak/boosting-adaboost-gradient-boost-and-xgboost-bdda87eed44e>)

Example –

Problem: Loan Approval Prediction

Goal: Predict whether a loan will be **Approved (+1)** or **Rejected (-1)**

Weak learner: Decision stump

Features:

- Income (Low / High)
- Credit Score (Poor / Good)

Dataset

Sample Income Credit Score Loan (y)

1	Low	Poor	-1
2	Low	Good	-1
3	High	Poor	+1
4	High	Good	+1
5	Low	Poor	+1

👉 This dataset is **not perfectly separable**, making it realistic.

Step 1: Initialize Sample Weights

Total samples $N = 5$

$$w_i = \frac{1}{5} = 0.2$$

Round 1 – Weak Learner 1

Choose best decision stump

Rule:

$$h_1(x) = \begin{cases} +1 & \text{if Income = High} \\ -1 & \text{if Income = Low} \end{cases}$$

Predictions

Sample True y Pred Correct? Weight

1	-1	-1	✓	0.2
2	-1	-1	✓	0.2
3	+1	+1	✓	0.2
4	+1	+1	✓	0.2
5	+1	-1	✗	0.2

✗ Weighted Error

$$\epsilon_1 = 0.2$$

☆ Learner Weight

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0.2}{0.2} \right) = \frac{1}{2} \ln(4) = 0.693$$

📦 Update Sample Weights

Misclassified sample (5):

$$w_5 = 0.2 \times e^{+0.693} = 0.4$$

Correct samples:

$$w = 0.2 \times e^{-0.693} = 0.1$$

Normalize Weights

$$\text{Sum} = 0.1 \times 4 + 0.4 = 0.8$$

Sample New Weight

1	0.125
2	0.125
3	0.125
4	0.125
5	0.500

Round 2 – Weak Learner 2

Focus shifts to **Sample 5**

New stump:

$$h_2(x) = \begin{cases} +1 & \text{if Credit Score = Poor} \\ -1 & \text{if Credit Score = Good} \end{cases}$$

Predictions

Sample True y Pred Correct? Weight

1	-1	+1	✗	0.125
2	-1	-1	✓	0.125
3	+1	+1	✓	0.125
4	+1	-1	✗	0.125
5	+1	+1	✓	0.500

✗ Weighted Error

$$\epsilon_2 = 0.125 + 0.125 = 0.25$$

☆ Learner Weight

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - 0.25}{0.25} \right) = 0.549$$

📦 Final Strong Classifier

$$H(x) = \text{sign}(0.693 \cdot h_1(x) + 0.549 \cdot h_2(x))$$

🔍 Example Prediction

New applicant:

Income = Low, Credit Score = Poor

- $h_1 = -1$
- $h_2 = +1$

$$H(x) = \text{sign}(-0.693 + 0.549) = \text{sign}(-0.144) = -1$$

➡ Loan Rejected

When to use AdaBoost

- When weak learners perform slightly better than random guessing
- When the dataset is small to medium-sized and relatively clean
- When you want to reduce bias using simple models like decision stumps

When not to use AdaBoost

- When the dataset contains heavy noise or many outliers
- When data is very large or highly complex, making training slow or unstable

Gradient Boosting Algorithm

Gradient Boosting is an ensemble learning algorithm that builds a strong predictive model by sequentially adding weak learners (typically shallow decision trees), where each new learner is trained to minimize the loss function by fitting the negative gradient (residual errors) of the current ensemble's predictions.

Why It's Called "Gradient" Boosting

Because each model:

- Learns the negative gradient of the loss
- Uses gradient descent in function space
- Optimizes any differentiable loss function

Step-by-Step Working of Gradient Boosting

◆ Step 1: Choose a loss function

The loss function defines what "error" means.

- **Regression:** Mean Squared Error (MSE)

$$L(y, \hat{y}) = (y - \hat{y})^2$$

- **Classification:** Log loss

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

◆ Step 2: Initialize the model

Start with a **constant prediction** that minimizes the loss.

For regression (MSE):

$$F_0(x) = \arg \min_c \sum (y_i - c)^2 = \text{mean}(y)$$

This is the **baseline model**.

◆ Step 3: Compute residuals (negative gradients)

At iteration t , compute the residuals:

$$r_i^{(t)} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

For MSE loss:

$$r_i^{(t)} = y_i - \hat{y}_i$$

These residuals represent **what the model still fails to explain**.

◆ Step 4: Train a weak learner on residuals

Train a **shallow decision tree** $h_t(x)$ to predict the residuals.

- Inputs: original features x
 - Targets: residuals $r_i^{(t)}$
-

◆ Step 5: Compute optimal step size (optional but important)

Find the best multiplier γ_t for the new tree:

$$\gamma_t = \arg \min_{\gamma} \sum L(y_i, F_{t-1}(x_i) + \gamma h_t(x_i))$$

This ensures the update moves in the **best direction**.

◆ Step 6: Update the ensemble model

$$F_t(x) = F_{t-1}(x) + \eta \cdot \gamma_t \cdot h_t(x)$$

Where:

- η = learning rate (shrinkage)
 - Controls overfitting
-

◆ Step 7: Repeat for T iterations

Repeat **Steps 3–6** until:

- Maximum trees reached
- Loss converges

- Validation error increases (early stopping)
-

5 Final Prediction

- **Regression:**

$$\hat{y} = F_T(x)$$

- **Classification:**
Apply sigmoid or softmax to convert scores to probabilities.

Example –

Problem: Predict House Rent (Regression)

We want to predict **monthly rent** based on **house size**.

Dataset (Small Real-World Dataset)

House Size (sq ft) Rent (₹ in thousands)

1	500	15
2	700	20
3	900	25
4	1100	28

Assumptions

- **Loss function:** Mean Squared Error (MSE)

$$L(y, \hat{y}) = (y - \hat{y})^2$$

- **Weak learners:** Decision stumps
 - **Learning rate:** $\eta = 0.5$
 - **Number of boosting rounds:** 2
-

◆ Step 1: Initialize the Model

For MSE loss, the best constant prediction is the **mean of target values**:

$$F_0 = \frac{15 + 20 + 25 + 28}{4} = 22$$

Initial prediction for all houses:

$$\hat{y} = 22$$

◆ **Step 2: Compute Residuals (Iteration 1)**

$$r_i = y_i - \hat{y}_i$$

House Actual Rent Prediction Residual

1	15	22	-7
2	20	22	-2
3	25	22	+3
4	28	22	+6

◆ **Step 3: Train Weak Learner 1 on Residuals**

Decision stump split:

Rule:

If Size $\leq 800 \Rightarrow$ predict - 4.5

If Size $> 800 \Rightarrow$ predict + 4.5

(These are mean residuals of each group.)

◆ **Step 4: Update Model (Iteration 1)**

$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$

House $h_1(x)$ New Prediction

1	-4.5	22 - 2.25 = 19.75
---	------	-------------------

House $h_1(x)$ New Prediction

2	-4.5	19.75
3	+4.5	$22 + 2.25 = 24.25$
4	+4.5	24.25

◆ Step 5: Compute Residuals (Iteration 2)

$$r_i = y_i - \hat{y}_i$$

House Actual New Prediction Residual

1	15	19.75	-4.75
2	20	19.75	+0.25
3	25	24.25	+0.75
4	28	24.25	+3.75

◆ Step 6: Train Weak Learner 2 on Residuals

Decision stump split:

Rule:

If Size $\leq 900 \Rightarrow$ predict - 1.25
If Size $> 900 \Rightarrow$ predict + 3.75

◆ Step 7: Update Model (Iteration 2)

$$F_2(x) = F_1(x) + \eta \cdot h_2(x)$$

House $h_2(x)$ Final Prediction

1	-1.25	$19.75 - 0.625 = 19.13$
2	-1.25	19.13
3	-1.25	$24.25 - 0.625 = 23.63$

House $h_2(x)$ Final Prediction

$$4 \quad +3.75 \quad 24.25 + 1.875 = 26.13$$

🚩 Final Gradient Boosting Model

$$F(x) = 22 + 0.5h_1(x) + 0.5h_2(x)$$

🔍 Example Prediction

New house: 1000 sq ft

- $h_1 = +4.5$
- $h_2 = +3.75$

$$\hat{y} = 22 + 0.5(4.5) + 0.5(3.75) = 26.125$$

➡ **Predicted rent $\approx$ ₹26,000**

ASSUMPTIONS OF LINEAR REGRESSION

LINEARITY

Assumption: Linear regression assumes a linear relationship between independent variables (features) and the dependent variable (target). Non-linear relationships lead to biased or inaccurate predictions.

How to detect:

1. Scatter Plots

- Plot each independent variable (X) against the dependent variable (Y).
- Look for a straight-line trend. Curved or wavy patterns indicate non-linearity.

2. Correlation Coefficient

- Compute Pearson correlation for numerical variables.
- Strong correlations (near ± 1) suggest a linear relationship, while weak correlations may indicate non-linearity or noise.

Independence of Errors

Assumption: Residuals (errors) should be independent of each other. Violations occur in time series or clustered data.

How to detect:

1. Residual Plots by Observation Order

- Plot residuals against the order of observations.
- Patterns or trends indicate correlation between errors.

Homoscedasticity

Assumption: Residuals should have constant variance across all levels of independent variables. Unequal spread is called heteroscedasticity.

How to detect:

1. Residuals vs. Fitted Values Plot

- Plot residuals on Y-axis and predicted values on X-axis.
- Spread should be roughly uniform; funnel or “fan-shaped” pattern → heteroscedasticity.

2. Breusch-Pagan Test / White Test

- Formal statistical tests to check for non-constant variance.

- Significant p-value → heteroscedasticity present.

Multicollinearity

Assumption: Independent variables should not be highly correlated. High multicollinearity inflates coefficient variances and makes interpretation unreliable.

How to detect:

1. Correlation Matrix

- Compute pairwise correlations among independent variables.
- High correlations (e.g., >0.8) indicate multicollinearity.

2. Variance Inflation Factor (VIF)

- For each predictor: $VIF_i = \frac{1}{1-R_i^2}$
- $VIF > 10$ (or sometimes >5) → problematic multicollinearity.

ASSUMPTIONS OF LOGISTIC REGRESSION

Linearity of Log-Odds

Assumption:

Logistic regression assumes that each continuous independent variable has a linear relationship with the log-odds (logit) of the dependent variable.

This is different from linear regression — the relationship is not linear with probability, only with the logit.

Why it matters:

If the relationship is not linear in the logit, the model may underestimate or overestimate probabilities, leading to poor predictions.

Methods to detect:

1. Box-Tidwell Test

- For each continuous predictor X , include an interaction term with its log: $X \times \ln(X)$.
- Fit the logistic regression including these terms.
- Significant interaction → linearity of log-odds is violated.

Independence of Observations

Assumption:

Observations should be independent; one observation should not influence another.

Violation occurs in repeated measurements, clustered data, or time-series, leading to biased standard errors and p-values.

Multicollinearity

Assumption:

Predictors should not be highly correlated with each other. High correlation inflates standard errors, making coefficients unstable and unreliable.

Methods to detect:

1. Correlation Matrix

- Compute pairwise correlations among independent variables.
- Correlation > 0.8 (absolute) is usually considered high.

2. Variance Inflation Factor (VIF)

- For each predictor, compute VIF.
- $VIF > 10 \rightarrow$ indicates multicollinearity problem.

ASSUMPTIONS OF KNN

Similarity Implies Proximity

Assumption:

KNN assumes that similar points in feature space belong to the same class (classification) or have similar values (regression). In other words, nearby points are more alike than distant points.

Why it matters:

If similar points are far apart, KNN will pick “wrong neighbours,” leading to misclassification or poor predictions.

Meaningful Distance Metric

Assumption:

KNN relies on a distance metric (Euclidean, Manhattan, Minkowski, cosine, etc.) to measure similarity.

The metric must accurately reflect real similarity; otherwise, neighbours are chosen incorrectly.

Why it matters:

If distance is meaningless (e.g., raw features with different scales or irrelevant features), KNN will perform poorly.

Locality of Decision Boundary

Assumption:

KNN assumes that local neighbourhoods are sufficient to determine the class or value.

The prediction for a point depends only on its k nearest neighbours, not on global patterns.

Why it matters:

If the decision boundary is highly complex or global patterns dominate, KNN may misclassify points because local neighbors are misleading.

ASSUMPTIONS OF NAÏVE BAYES

Assumptions of Naive Bayes



Conditional Independence of Features

Naive Bayes assumes that features are **independent** of each other given the **class label**. ❤

Often violated, yet the model performs well in many real-world cases.



Correct Feature Distribution

The model assumes features follow a specific probability distribution (Gaussian, Multinomial, or Bernoulli).

Using the wrong distribution leads to incorrect probability estimation.



Equal Contribution of Features

Every feature is assumed to contribute independently and **equally** to the prediction.

Feature interactions are ignored, which can limit expressiveness.



Representative Training Data

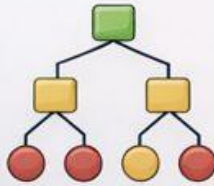
Naive Bayes assumes the training data *accurately reflects real-world class distributions*.

Biased or skewed datasets can lead to poor probability estimates.



ASSUMPTIONS OF DECISION TREES

Assumptions of Decision Trees



1. Recursive Partitioning Assumption

Decision trees assume that data can be **recursively partitioned into subsets** based on **simple, decision rules applied sequentially**.

This means each split is made based on the **most informative** feature at the current node, creating homogenous subgroups in a top-down manner.

The quality of splits is assessed at each step, leading to a final tree where each path from root to leaf represents a decision rule.



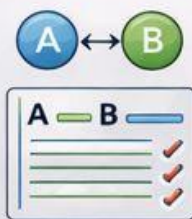
2. Predictive Feature Assumption

It is assumed that some **features** in the dataset have a **strong predictive power** for the target variable.

These features allow the tree to make informative splits that reduce impurity effectively.

If the features lack **predictive power**, the decision tree will struggle to **distinguish** between classes, leading to a tree with poor performance.

In such cases, the model may perform no better than **random guessing**.



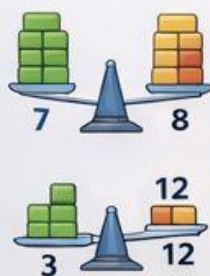
3. Minimal Correlated Predictors

Decision trees assume that predictors are not too **highly correlated** with each other.

Strongly correlated predictors may lead the tree to split on one feature, disregarding the other redundant, correlated feature.

This can limit the tree's ability to identify the most relevant predictors.

To address this, one can consider using feature **importance metrics** or ensemble methods like **Random Forests** that reduce the impact of correlated predictors.



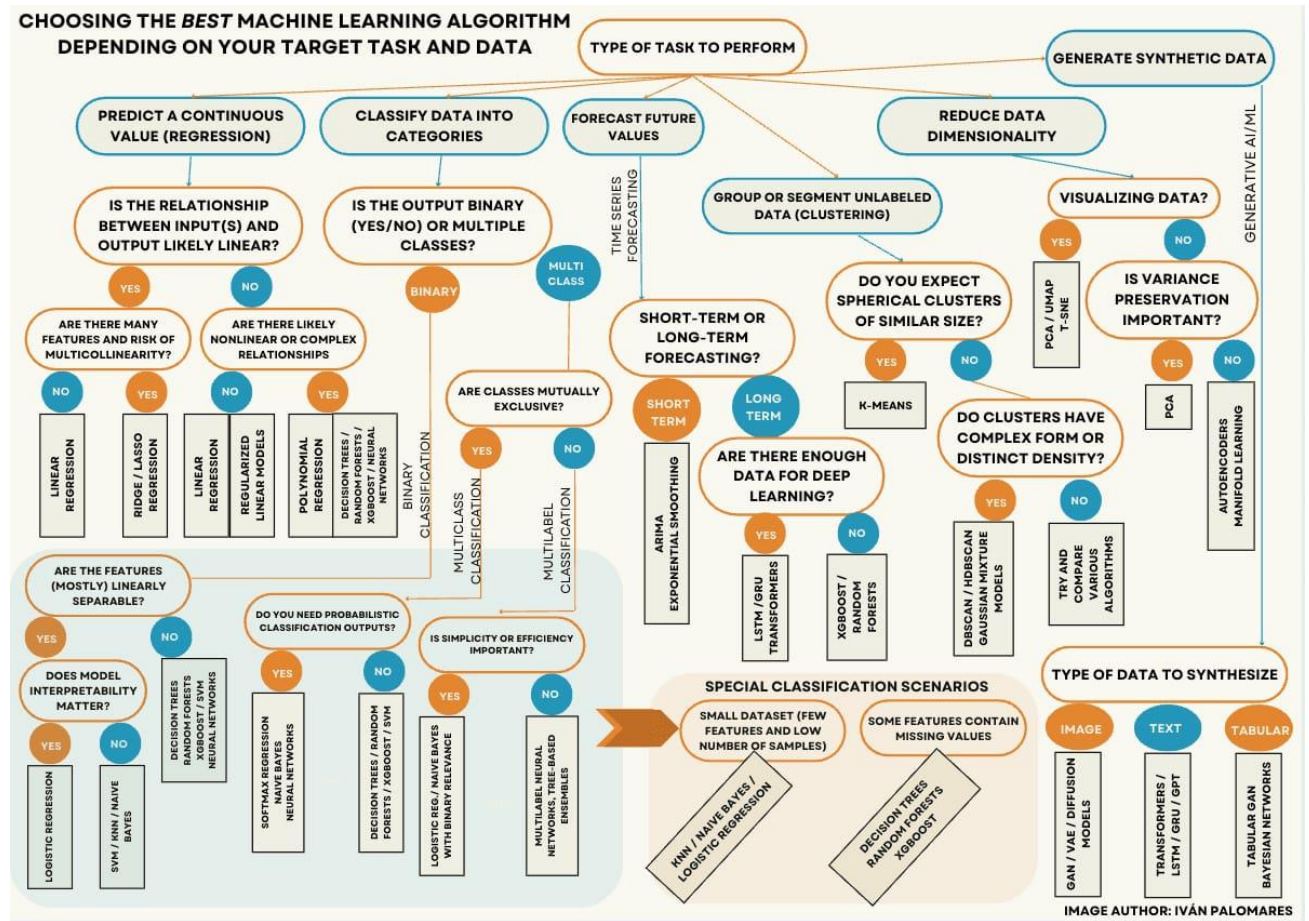
4. Balanced Binary Splits Often Preferred

Although not strictly necessary, **decision trees** often assume **balanced splits** where child nodes are of relatively equal size.

Balanced splits tend to create more stable and **interpretable** trees as they avoid overly imbalanced partitions which can lead to **overfitting**.

For highly imbalanced classes, this assumption is relaxed, but careful tuning of splitting criteria and tree depth is needed to prevent the tree from favoring the majority class.

WHEN TO USE WHICH ML MODEL



WHEN TO USE WHICH PERFORMANCE METRICS

Metric	When to Prioritize	Business or Real-World Impact	Cost of FP	Cost of FN	Example Scenario	When Not to Use
Accuracy	When class distribution is balanced and both error types are equally important	Quick overall check of model correctness	Moderate	Moderate	Email classification: ~50% spam, ~50% non-spam	Imbalanced datasets (rare events), or when FP/FN have very different costs

Metric	When to Prioritize	Business or Real-World Impact	Cost of FP	Cost of FN	Example Scenario	When Not to Use
Precision	When false positives are costly	Avoids unnecessary or harmful actions based on wrong positive predictions	High	Low	Medical testing for a rare disease: misdiagnosing healthy people	When missing positives is more critical than avoiding false alarms
Recall	When false negatives are costly	Ensures that important positive cases are not missed	Low	High	Fraud detection: missing fraudulent transactions	When false positives are very costly and must be minimized
F1 Score	When classes are imbalanced and both FP & FN matter	Balanced view of performance in rare-event detection	Medium	Medium	Predicting defective products in manufacturing: catch defects but avoid unnecessary re-inspections	When one type of error dominates business cost and you want to prioritize either precision or recall specifically